PACKET SNIFFING AND SPOOFING

CS44500 Computer Security

How Packets Are Received

- NIC (Network Interface Card) is a physical or logical link between a machine and a network.
- Each NIC has a MAC address.
- Every NIC on the network will hear all the frames on the wire.
- NIC checks the destination address for every packet, if the address matches the cards MAC address, it is further copied into a buffer in the kernel.

Promiscuous Mode

- The frames that are not destined to a given NIC are usually discarded
- When operating in promiscuous mode, NIC passes every frame received from the network to the kernel
- If a sniffer program is registered with the kernel, it will be able to see all the packets
- In Wi-Fi, it is called Monitor Mode

Packet Sniffing

Packet sniffing describes the process of capturing live data as they flow across a network

Let’s first see how computers receive packets.

Receiving Packets Using Socket

Create the socket

Provide information about server

Receive packets

```
// Step ①
int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

// Step ②
memset((char *) &server, 0, sizeof(server));
server.sin_family = AF_INET;
server.sin_addr.s_addr = htonl(INADDR_ANY);
server.sin_port = htons(9090);

if (bind(sock, (struct sockaddr *) &server, sizeof(server)) < 0)
    error("ERROR on binding");

// Step ③
while (1) {
    bzero(buf, 1500);
    recvfrom(sock, buf, 1500-1, 0,
                (struct sockaddr *) &client, &clientlen);
    printf("%s\n", buf);
}
```

Receiving Packets Using Raw Socket

Creating a raw socket

Capture all types of packets

```
// Create the raw socket
int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));   ①

// Turn on the promiscuous mode.
mr.mr_type = PACKET_MR_PROMISC;                              ②
setsockopt(sock, SOL_PACKET, PACKET_ADD_MEMBERSHIP, &mr,     ③
                 sizeof(mr));

// Getting captured packets
while (1) {
    int data_size=recvfrom(sock, buffer, PACKET_LEN, 0,      ④
                &saddr, (socklen_t*)sizeof(saddr));
    if(data_size) printf("Got one packet\n");
}
```

Enable the promiscuous mode

Wait for packets

Limitation of the Approach

- This program is not portable across different operating systems.
- Setting filters is not easy.
- The program does not explore any optimization to improve performance.
- The PCAP library was thus created.
  - It still uses raw sockets internally, but its API is standard across all platforms. OS specifics are hidden by PCAP's implementation.
  - Allows programmers to specify filtering rules using human readable Boolean expressions.

Packet Sniffing Using the pcap API

Initialize a raw socket, set the network device into promiscuous mode.

```
char filter_exp[] = "ip proto icmp";
```

Filter

```
// Step 1: Open live pcap session on NIC with name eth3
handle = pcap_open_live("eth3", BUFSIZ, 1, 1000, errbuf);  ①

// Step 2: Compile filter_exp into BPF psuedo-code
pcap_compile(handle, &fp, filter_exp, 0, net);             ②
pcap_setfilter(handle, &fp);                               ③

// Step 3: Capture packets
pcap_loop(handle, -1, got_packet, NULL);                   ④
```

Invoke this function for every captured packet

```
void got_packet(u_char *args, const struct pcap_pkthdr *header,
        const u_char *packet)
{
   printf("Got a packet\n");
}
```

Scapy Example 1

```
#!/usr/bin/python3

from scapy.all import *

pkt = sniff(iface='enp0s3',
            filter='icmp or udp',
            count=10)

pkt.summary()
```

Scapy Example 2

```
#!/usr/bin/python3

from scapy.all import *

def process_packet(pkt):
     #hexdump(pkt)
     pkt.show()
     print("-----------------------------------")

f = 'udp and dst portrange 50-55 or icmp'

sniff(iface='enp0s3', filter = f, prn=process_packet)
```

A Sniffer Example

```
def process_packet(pkt):
    if pkt.haslayer(IP):
       ip =  pkt[IP]
       print("IP: {} --> {}".format(ip.src, ip.dst))

    if pkt.haslayer(TCP):
       tcp = pkt[TCP]
       print("    TCP  port: {} --> {}".format(tcp.sport, tcp.dport))

    elif pkt.haslayer(UDP):
       udp = pkt[UDP]
       print("    UDP  port: {} --> {}".format(udp.sport, udp.dport))

    elif pkt.haslayer(ICMP):
       icmp = pkt[ICMP]
       print("    ICMP type: {}".format(icmp.type))

    else:
       print("    Other protocol")


sniff(iface='enp0s3', filter='ip', prn=process_packet)
```

Packet Spoofing

- When some critical information in the packet is forged, we refer to it as packet spoofing.
- Many network attacks rely on packet spoofing.
- Let’s see how to send packets without spoofing.

Sending Packets Without Spoofing

```
void main()
{
    struct sockaddr_in dest_info;
    char *data = "UDP message\n";

    // Step 1: Create a network socket
    int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    // Step 2: Provide information about destination.
    memset((char *) &dest_info, 0, sizeof(dest_info));
    dest_info.sin_family = AF_INET;
    dest_info.sin_addr.s_addr = inet_addr("10.0.2.5");
    dest_info.sin_port = htons(9090);

    // Step 3: Send out the packet.
    sendto(sock, data, strlen(data), 0,
                 (struct sockaddr *)&dest_info, sizeof(dest_info));
    close(sock);
}
```

Testing: Use the `netcat` (`nc`) command to run a UDP server on `10.0.2.5`. We then run the program on the left from another machine. We can see that the message has been delivered to the server machine:

```
seed@Server(10.0.2.5):$ nc -luv 9090
Connection from 10.0.2.6 port 9090 [udp/*] accepted
UDP message
```

Spoofing Packets Using Raw Sockets

```
/*************************************************************
  Given an IP packet, send it out using a raw socket.
*************************************************************/
void send_raw_ip_packet(struct ipheader* ip)
{
    struct sockaddr_in dest_info;
    int enable = 1;

    // Step 1: Create a raw network socket.
    int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);

    // Step 2: Set socket option.
    setsockopt(sock, IPPROTO_IP, IP_HDRINCL,
                     &enable, sizeof(enable));

    // Step 3: Provide needed information about destination.
    dest_info.sin_family = AF_INET;
    dest_info.sin_addr = ip->iph_destip;

    // Step 4: Send the packet out.
    sendto(sock, ip, ntohs(ip->iph_len), 0,
           (struct sockaddr *)&dest_info, sizeof(dest_info));
    close(sock);
}
```

We use *setsockopt()* to enable *IP_HDRINCL* on the socket.

For raw socket programming, since the destination information is already included in the provided IP header, we do not need to fill all the fields

Since the socket type is raw socket, the system will send out the IP packet as is.

Spoofing ICMP Packets

```
#!/usr/bin/python3
from scapy.all import *

print("SENDING SPOOFED ICMP PACKET.........")
ip = IP(src="1.2.3.4", dst="93.184.216.34")
icmp = ICMP()
pkt = ip/icmp
pkt.show()
send(pkt,verbose=0)
```

ICMP (Internet Control Message Protocol)

Spoofing UDP Packets

```
#!/usr/bin/python3
from scapy.all import *

print("SENDING SPOOFED UDP PACKET.........")
ip = IP(src="1.2.3.4", dst="10.0.2.69") # IP Layer
udp = UDP(sport=8888, dport=9090)       # UDP Layer
data = "Hello UDP!\n"                   # Payload
pkt = ip/udp/data
pkt.show()
send(pkt,verbose=0)
```

Sniffing and Then Spoofing

- In many situations, we need to capture packets first, and then spoof a response based on the captured packets.
- Procedure (using UDP as example)
  - Use PCAP API to capture the packets of interests
  - Make a copy from the captured packet
  - Replace the UDP data field with a new message and swap the source and destination fields
  - Send out the spoofed reply

Sniff Request and Spoof Reply: Code

```
def spoof_pkt(pkt):
  if ICMP in pkt and pkt[ICMP].type == 8:
     print("Original Packet.........")
     print("Source IP : ", pkt[IP].src)
     print("Destination IP :", pkt[IP].dst)

     ip = IP(src=pkt[IP].dst, dst=pkt[IP].src,
             ihl=pkt[IP].ihl, ttl = 99)
     icmp = ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
     data = pkt[Raw].load
     newpkt = ip/icmp/data

     print("Spoofed Packet.........")
     print("Source IP : ", newpkt[IP].src)
     print("Destination IP :", newpkt[IP].dst)

     send(newpkt,verbose=0)

pkt = sniff(iface = 'br-54cdc1101b1a',
            filter = 'icmp and src host 10.9.0.5',
            prn = spoof_pkt)
```

In ICMP, a "type" of 8 corresponds to an Echo Request message, which is commonly used for pinging a remote host.

Swap source and destination

Echo Reply (Type o)

Example: implement ping

```
#!/usr/bin/python3
from scapy.all import *

ip = IP(dst="8.8.8.8")
icmp = ICMP()
pkt = ip/icmp
reply = sr1(pkt)
print("ICMP reply .........")
print("Source IP : ", reply[IP].src)
print("Destination IP :", reply[IP].dst)
```

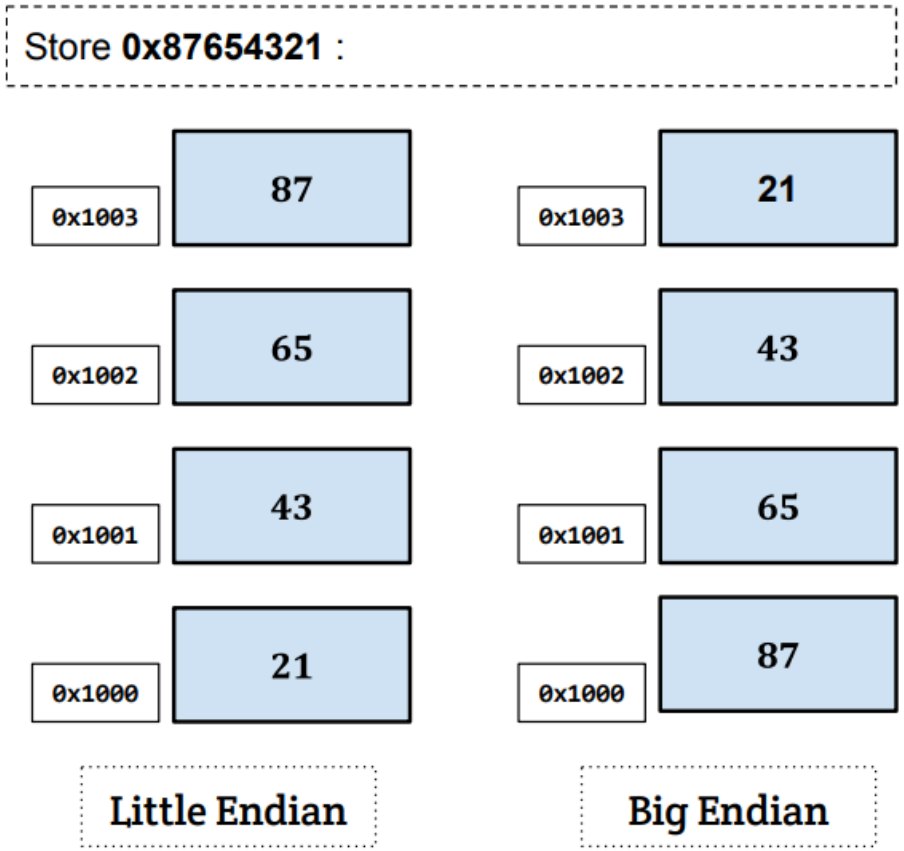
Traceroute Code

```
b = ICMP()
a = IP()
a.dst = '93.184.216.34'

TTL = 3
a.ttl = TTL
h = sr1(a/b, timeout=2, verbose=0)
if h is None:
    print("Router: *** (hops = {})".format(TTL))
else:
    print("Router: {} (hops = {})".format(h.src, TTL))
```

Endianness

- Endianness: a term that refers to the order in which a given multi-byte data item is stored in memory.
  - **Little Endian**: store the most significant byte of data at the highest address
  - **Big Endian**: store the most significant byte of data at the lowest address



Endianness In Network Communication

- Computers with different byte orders will "misunderstand" each other.
  - Solution: agree upon a common order for communication
  - This is called "network order", which is the same as big endian order
- All computers need to convert data between "host order" and "network order" .

Macro	Description
`htons()`	Convert unsigned short integer from host order to network order.
`htonl()`	Convert unsigned integer from host order to network order.
`ntohs()`	Convert unsigned short integer from network order to host order.
`ntohl()`	Convert unsigned integer from network order to host order.

Summary

- Packet sniffing
  - Using raw socket
  - Using PCAP APIs
- Packet spoofing using raw socket
- Sniffing and the spoofing
- Endianness